

LAPORAN TAHAP 2 – IMPLEMENTASI BOOK SERVICE

Sistem Peminjaman Buku Berbasis Arsitektur Microservices

A. IDENTITAS IMPLEMENTASI

NAMA PROYEK Sistem Peminjaman Buku	TAHAPAN Tahap 2 – Implementasi Microservice
KOMPONEN UTAMA Book Service	STACK TEKNOLOGI Node.js, Express.js, REST API, JSON
PORT LAYANAN 3001	PENYIMPANAN (STORAGE) data/books.json

Tujuan: Book Service dikembangkan untuk menangani pengelolaan data katalog buku dan status ketersediaan buku dalam arsitektur microservice. Service ini menyediakan API yang dapat digunakan secara independen oleh Frontend dan Loan Service.

B. ALUR PENGEMBANGAN BOOK SERVICE

Pengembangan dilakukan secara bertahap menggunakan AI Coding Tool melalui alur verifikasi ganda (Human-in-the-loop):



*Setiap hasil generasi AI diperiksa secara ketat oleh pengembang agar tetap berada dalam cakupan rancangan Tahap 1.

C. PROMPT 1 – ANALISIS BOOK SERVICE

Tujuan Prompt

Prompt pertama digunakan untuk meminta AI menganalisis kebutuhan serta rancangan arsitektur Book Service sebelum penulisan kode program dimulai.

Teks Prompt 1

Saya sedang mengembangkan Sistem Peminjaman Buku dari arsitektur monolith menjadi arsitektur microservice. Saya bertanggung jawab hanya pada bagian Book Service. Pada tahap ini saya belum ingin membuat kode. Saya ingin melakukan analisis dan perancangan terlebih dahulu. Gunakan spesifikasi berikut sebagai sumber kebenaran:

- Arsitektur sistem terdiri dari:
 - Frontend
 - Book Service
 - Loan Service
- Book Service berjalan pada port 3001.
- Tanggung jawab Book Service:
 - Mengelola data katalog buku.
 - Menyediakan informasi daftar buku.
 - Menyediakan detail buku berdasarkan ID.
 - Mengelola status ketersediaan buku.
- Storage Book Service menggunakan file JSON:
data/books.json
- Status buku hanya memiliki dua nilai:
 - available
 - borrowed
- API Book Service yang direncanakan:

- GET /api/books
 - GET /api/books/:id
 - PATCH /api/books/:id/status
7. Endpoint PATCH /api/books/:id/status nantinya digunakan oleh Loan Service untuk mengubah status buku setelah proses peminjaman berhasil.
8. Frontend tidak mengakses file JSON secara langsung. Frontend berkomunikasi dengan Book Service melalui REST API.
9. Loan Service bertanggung jawab terhadap transaksi peminjaman, batas maksimal 3 buku aktif, tanggal peminjaman, dan tanggal jatuh tempo. Jangan memasukkan tanggung jawab tersebut ke Book Service.
10. Sistem tidak membutuhkan:
- Auth Service
 - Admin Service
 - database
 - fitur pengembalian buku
 - dashboard
 - pencarian buku
 - filter buku
 - notifikasi
 - fitur lain di luar spesifikasi.
- Teknologi yang direncanakan:
- Node.js
 - Express.js
 - REST API
 - JSON file sebagai storage.
- Analisis yang saya butuhkan:
- A. Jelaskan tanggung jawab Book Service dan batas tanggung jawabnya terhadap Frontend dan Loan Service.
- B. Tentukan struktur folder dan file minimal yang dibutuhkan untuk Book Service beserta fungsi masing-masing file.
- C. Analisis kebutuhan setiap endpoint:
- method
 - URL
 - fungsi
 - request yang dibutuhkan
 - response yang dihasilkan
 - kemungkinan error.
- D. Jelaskan bagaimana Book Service membaca dan memperbarui data pada books.json.
- E. Jelaskan aturan validasi yang diperlukan, khususnya:
- ID buku tidak ditemukan.
 - status tidak valid.
 - perubahan status buku.
- F. Jelaskan alur komunikasi antara Loan Service dan Book Service ketika status buku perlu diubah.
- G. Identifikasi potensi masalah atau kesalahan implementasi yang perlu diperhatikan saat coding nanti.
- H. Berikan contoh struktur data books.json yang sederhana.
- Jangan membuat kode program pada tahap ini.
- Jangan menambahkan fitur yang tidak disebutkan.
- Jika ada bagian yang belum dapat ditentukan dari spesifikasi, tandai sebagai "perlu diputuskan" daripada membuat asumsi sendiri.
- Susun jawaban secara terstruktur agar hasil analisis ini dapat saya review terlebih dahulu sebelum masuk ke tahap implementasi.

D. HASIL AI 1 – ANALISIS SERVICE

AI memberikan rekomendasi struktur folder modular beserta pembagian tanggung jawab komponen sebagai berikut:

```
book- service/
├── data/
│   └── books. j son
├── src/
│   ├── app. j s
│   ├── server. j s
│   ├── routes/
│   │   └── bookRoutes. j s
│   ├── controllers/
│   │   └── bookController. j s
└── package. j son
```

- `books. j son` : Penyimpanan data master buku lokal.
- `bookRoutes. j s` : Penanganan routing REST API.
- `bookController. j s` : Pengolahan bisnis logika (baca/tulis data).
- `app. j s` : Konfigurasi Express & middleware.
- `server. j s` : Entry point untuk menjalankan server pada port 3001 .

E. HUMAN REVIEW 1

Hasil analisis AI diperiksa terhadap rancangan awal Tahap 1 . AI berhasil membatasi cakupan tanpa menambah fitur di luar spesifikasi. Keputusan teknis disepakati sebagai berikut:

Aspek Teknis	Keputusan Final (Source of Truth)
Format ID Buku	String incremental terstruktur (contoh: b1 , b2 , b3)
Storage & Port	JSON file pada data/books. j son Port 3001
Status Valid	Hanya bernilai available atau borrowed
Responsivitas Error	Validation status (HTTP 400), Not Found (HTTP 404)
Isolasi Fitur	Tanpa Authentication, Admin, Search, atau Database SQL/NoSQL

F. PROMPT 2 – IMPLEMENTASI BOOK SERVICE

Berdasarkan hasil analisis Book Service yang sudah saya review, sekarang bantu saya mengimplementasikan Book Service.

Gunakan keputusan berikut sebagai sumber kebenaran dan jangan mengubahnya:

ARSITEKTUR:

- Sistem terdiri dari Frontend, Book Service, dan Loan Service.
- Book Service berjalan pada port 3001.
- Book Service bertanggung jawab atas katalog buku dan status ketersediaan buku.
- Book Service tidak bertanggung jawab atas transaksi peminjaman, batas maksimal 3 buku, tanggal peminjaman, atau tanggal jatuh tempo.

TEKNOLOGI:

- Node.js
- Express.js
- REST API
- JSON file sebagai storage
- Tidak menggunakan database.

STORAGE:

- File: data/books.json
- ID buku menggunakan String, dengan format seperti "b1", "b2", "b3".

- Status buku hanya boleh:
- "available"
- "borrowed"

STRUKTUR YANG DIGUNAKAN:

```
book-service/
├── data/
│   └── books.json
├── src/
│   ├── app.js
│   ├── server.js
│   └── routes/
│       ├── bookRoutes.js
│       └── controllers/
│           └── bookController.js
└── package.json
```

TANGGUNG JAWAB FILE:

- app.js: konfigurasi Express, middleware, dan routing.
- server.js: menjalankan server pada port 3001.
- bookRoutes.js: mendefinisikan endpoint Book Service.
- bookController.js: menangani pembacaan/penulisan books.json, validasi, dan response.
- books.json: menyimpan data katalog buku.

API YANG WAJIB DIIMPLEMENTASIKAN:

1. GET /api/books
 - Mengembalikan seluruh data buku.
 - Response sukses: HTTP 200.
2. GET /api/books/:id
 - Mengembalikan satu buku berdasarkan ID.
 - Jika ID tidak ditemukan: HTTP 404.
3. PATCH /api/books/:id/status
 - Mengubah status buku.
 - Request body:


```
{
  "status": "available"
}
```

 atau


```
{
  "status": "borrowed"
}
```
 - Jika status selain "available" atau "borrowed": HTTP 400.
 - Jika ID buku tidak ditemukan: HTTP 404.
 - Jika pembacaan/penulisan file gagal: HTTP 500.

ATURAN:

- Gunakan express.json() agar dapat membaca request body JSON.
- Gunakan async/await dan penanganan error yang sesuai untuk operasi file.
- Jangan menambahkan authentication, authorization, admin, database, search, filter, return book, dashboard, notification, atau fitur lainnya.
- Jangan membuat endpoint tambahan yang tidak diperlukan.
- Jangan membuat Loan Service atau Frontend.
- Jangan mengubah port 3001.
- Jangan mengubah struktur tanggung jawab service.

Untuk tahap ini:

1. Buat seluruh file yang diperlukan.
2. Tampilkan struktur folder akhirnya.
3. Berikan isi kode setiap file.
4. Jelaskan secara singkat fungsi setiap file.
5. Jelaskan cara menjalankan Book Service.
6. Berikan contoh request dan response untuk ketiga endpoint.
7. Sertakan penanganan error yang sesuai.

Setelah memberikan kode, jangan menganggap implementasi sudah benar. Kode akan saya jalankan dan review secara manual melalui testing API.

G. HASIL AI 2 – KODE IMPLEMENTASI

1. Struktur File package.json

```
{
  "name": "book-service",
  "version": "1.0.0",
```

2. Data Awal (data/books.json)

```
{
  "id": "b1", "title": "Pemrograman Web dengan Node.js", "author": "Penulis A", "status": "av",
  "id": "b2", "title": "Arsitektur Microservices", "author": "Penulis B", "status": "availabl",
  "id": "b3", "title": "Panduan REST API", "author": "Penulis C", "status": "borrowed" }
```

3. Ringkasan Modul Utama (src/)

- `controllers/bookController.js` : Membaca dan memperbarui berkas JSON menggunakan `fs.promises`. Mengimplementasikan handler `getAllBooks`, `getBookById`, dan `updateBookStatus`.
- `routes/bookRoutes.js` : Memetakan endpoint REST API ke fungsi pengendali di controller.
- `app.js` : Inisialisasi Express, Express JSON parser, dan pemetaan ke rute `/api/books`.
- `server.js` : Menjalankan aplikasi pada port 3001.

H. HUMAN REVIEW 2

Parameter Evaluasi	Status Review	Catatan
Node.js & Express Architecture	SESUAI	Struktur CommonJS berjalan dengan tepat
Cakupan Endpoint	SESUAI	Mengakomodasi GET all, GET by ID, & PATCH status
Handling Validation (400 / 404)	SESUAI	Status selain available/borrowed ditolak
Isolasi Microservice	SESUAI	Tidak ada dependensi ke service lain/db luar

I. INSTALASI DAN EKSEKUSI SERVICE

Pengujian dilakukan pada lingkungan lokal pengembang dengan mengeksekusi perintah:

```
# 1. Instalasi Dependensi
npm.cmd install
```

```
# 2. Menjalankan Service
npm.cmd start
```

Service berhasil aktif dan siap menerima request HTTP pada `http://localhost:3001`.

J. PENGUJIAN API (THUNDER CLIENT)

Pengujian fungsionalitas REST API dilakukan secara menyeluruh:

- Test Case 1 (GET /api/books):** Berhasil menampilkan 3 data buku master. **PASS**
- Test Case 2 (GET /api/books/b1):** Mengembalikan spesifik objek buku `b1`. **PASS**
- Test Case 3 (PATCH status -> borrowed):** Mengubah status `b1` menjadi borrowed. **PASS**

- **Test Case 4 (PATCH status -> available):** Mengembalikan status `b1` ke available. PASS

A. CATATAN IMPLEMENTASI & TROUBLESHOOTING

Saat instalasi awal, sempat terjadi kesalahan eksekusi akibat berkas kontroler yang belum tersalin sepenuhnya pada direktori lokal. Kendala ini dikategorikan sebagai **masalah teknis pemindahan file lokal** dan bukan merupakan kesalahan logika/kode dari AI. Masalah terselesaikan penuh setelah penyalinan ulang dilakukan.

B. VERIFIKASI AKHIR (REGRESSION TESTING)

Endpoint API	Skenario Pengujian Regression	Status Akhir
GET /api/books	Pengambilan seluruh katalog pasca-update	PASS
GET /api/books/b1	Pemeriksaan integritas properti ID <code>b1</code>	PASS
PATCH /api/books/b1/status	Pembaruan ketersediaan ke <code>borrowed</code>	PASS
PATCH /api/books/b1/status	Pengembalian status ketersediaan ke <code>available</code>	PASS

C. GIT DAN KOLABORASI REMOTE

Seluruh pengerjaan dikelola menggunakan Git Flow versi ringan:

```
# Membuat branch fitur baru
git checkout -b feature/book-service git
add book-service/
git commit -m "feat: add book service"

# Push ke remote repository & membuat Pull Request git
push -u origin feature/book-service
```

D. HUMAN REVIEW AKHIR & KESIMPULAN

Book Service terbukti stabil, independen, dan terisolasi dengan baik. Service siap diintegrasikan dengan **Loan Service** pada Tahap 3.

